

GitHub Source Code Link:

https://github.com/chirayus20/Terraform_Chirayu

Part 1: Deploy Both Flask and Express on a Single EC2 Instance

Objective: Deploy both the Flask backend and the Express frontend on a single EC2 instance using Terraform.

1. Provision the EC2 instance with Terraform.
2. **Configure the instance with:** a user-data script or a configuration management tool (e.g., Ansible or Cloud-Init) to install dependencies (Python, Node.js), plus scripts to start the Flask backend and the Express frontend.
3. Ensure both applications run on different ports (e.g., Flask on port 5000 and Express on port 3000).

Expected Deliverables

4. Terraform configuration files (`main.tf`, `variables.tf`, etc.).
5. A working EC2 instance with Flask and Express running and accessible via the instance's public IP.

Part 2: Deploy Flask and Express on Separate EC2 Instances

Objective: Deploy the Flask backend and the Express frontend on two separate EC2 instances using Terraform.

6. **Provision two EC2 instances using Terraform:** one for the Flask backend and one for the Express frontend.
7. **Configure security groups to:** allow communication between the two instances and expose both applications to the internet on their respective ports.
8. **Configuration:** Use Terraform to define networking resources such as VPC, subnets, and security groups. Use user-data scripts to automate installation and startup of both applications.

Expected Deliverables

9. Terraform configuration files.
10. Two working EC2 instances: one running Flask and one running Express.
11. Security groups configured to allow proper communication and public access.

Part 3: Deploy Flask and Express Using Docker and AWS Services

Objective: Deploy Flask and Express as Docker containers using AWS ECR, ECS, and VPC with Terraform.

12. **ECR:** Use Terraform to create two Elastic Container Registry (ECR) repositories — one for the Flask backend and one for the Express frontend. Build Docker images for both and push them to their respective ECR repositories.
13. **VPC:** Use Terraform to create a VPC with subnets, route tables, and security groups.
14. **ECS:** Use Terraform to set up an ECS cluster and create two ECS services — one for Flask and one for Express — using ECS Fargate or the EC2 launch type to deploy the containers.
15. **Load Balancer:** Use Terraform to provision an Application Load Balancer (ALB) and configure ALB listeners to route requests to the appropriate ECS service.

Expected Deliverables

16. Terraform configuration files for ECR, ECS, VPC, and ALB setup.
17. Docker images pushed to ECR.
18. ECS services running and accessible via the ALB.

General Requirements

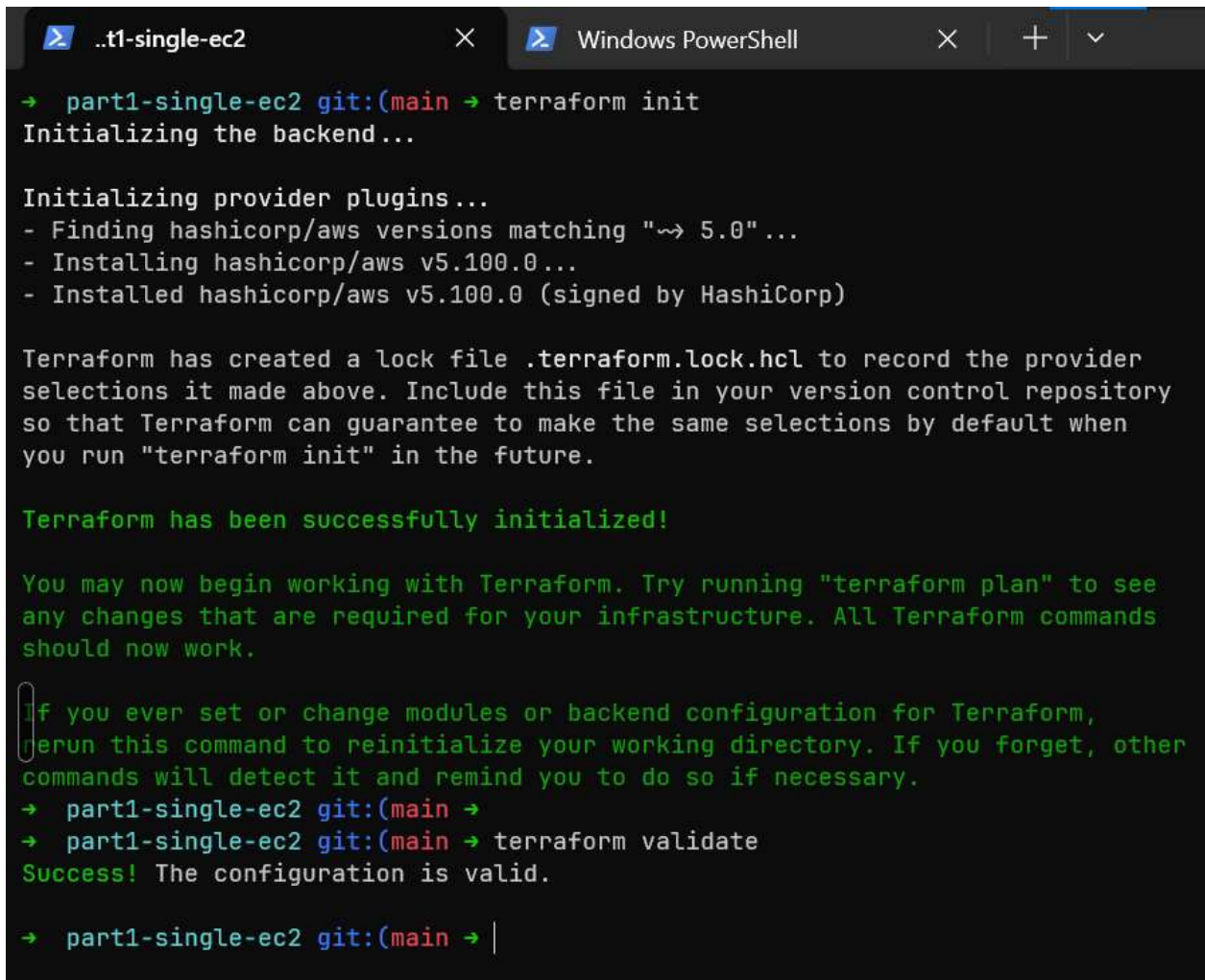
19. Ensure all Terraform configurations follow best practices, including the use of `variables.tf` and `outputs.tf`.
20. Use Terraform State (e.g., an `s3` backend) for state management.
21. Use `terraform plan` and `terraform apply` to validate and deploy infrastructure.
22. Document the deployment process in a `README.md` file.

Explanation: Part1-single-ec2

Step:1 Initialize Terraform Directory and Validate the Terraform Configuration.

Command:

- terraform init
- terraform validate



```
..t1-single-ec2  Windows PowerShell
→ part1-single-ec2 git:(main → terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0" ...
- Installing hashicorp/aws v5.100.0 ...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
→ part1-single-ec2 git:(main →
→ part1-single-ec2 git:(main → terraform validate
Success! The configuration is valid.

→ part1-single-ec2 git:(main → |
```

Initialize the working directory and install the AWS provider.

Validate the configuration files to ensure the syntax and internal consistency are correct.

Step 3: Run Terraform Plan

Command:

→ terraform plan

```
+ to_port      = 8000
# (1 unchanged attribute hidden)
},
+ {
+   cidr_blocks = [
+     "0.0.0.0/0",
+   ]
+   from_port    = 9000
+   ipv6_cidr_blocks = []
+   prefix_list_ids = []
+   protocol      = "tcp"
+   security_groups = []
+   self          = false
+   to_port      = 9000
+   # (1 unchanged attribute hidden)
+ },
+ ],
+ name           = "flask-express-sg"
+ name_prefix    = (known after apply)
+ owner_id       = (known after apply)
+ revoke_rules_on_delete = false
+ tags_all       = (known after apply)
+ vpc_id         = (known after apply)
}

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_instance_id = (known after apply)
+ ec2_public_ip   = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
+ part1-single-ec2 git:(main) →
```

Step 4: Secure Variable Configuration

Export sensitive database credentials as **TF_VAR_** environment variables to prevent exposing secrets in terminal logs or version control.

```
..t1-single-ec2  Windows PowerShell

→ part1-single-ec2 git:(main) → export TF_VAR_db_user="XXXXXXXXXX"
→ part1-single-ec2 git:(main) → export TF_VAR_db_password="XXXXXXXXXX"
→ part1-single-ec2 git:(main) → export TF_VAR_db_cluster="XXXXXXXXXX"
→ part1-single-ec2 git:(main) → export TF_VAR_db_name="XXXXXXXXXX"
→ part1-single-ec2 git:(main) →
```

Step 5: Provision Infrastructure on AWS

Provision the security group and EC2 instance on AWS using the exported configuration.

Command:

→ **terraform apply**

```
Changes to Outputs:
+ ec2_instance_id = (known after apply)
+ ec2_public_ip   = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

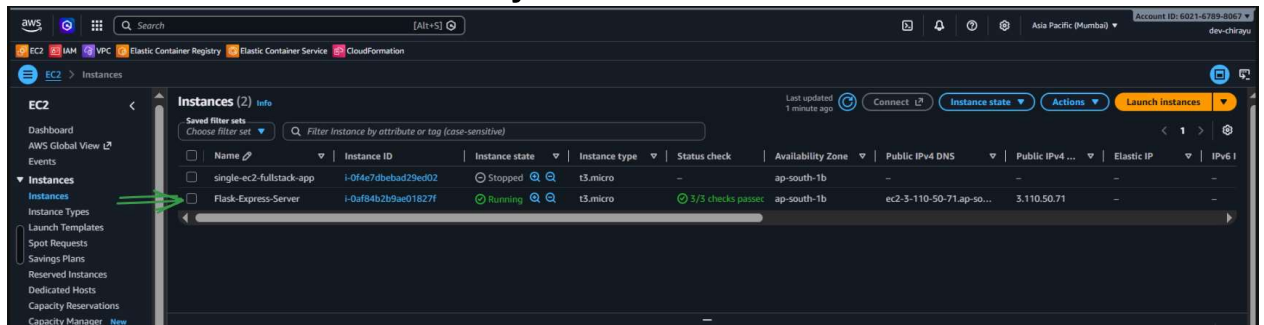
aws_security_group.app_sg: Creating...
aws_security_group.app_sg: Creation complete after 3s [id=sg-0f51689b6972c32b0]
aws_instance.app_server: Creating...
aws_instance.app_server: Still creating... [00m10s elapsed]
aws_instance.app_server: Creation complete after 13s [id=i-0af84b2b9ae01827f]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

ec2_instance_id = "i-0af84b2b9ae01827f"
ec2_public_ip   = "3.110.50.71"
→ part1-single-ec2 git:(main → |
```

EC2 Instance created successfully.



Step 6: Troubleshooting & Backend Environment Configuration

Problem:

The Flask backend service failed to start on port 9000, causing a DNS lookup failure (`_mongodb._tcp.none`) due to a missing `/home/ubuntu/app/backend/.env` file.

```
DB_NAME=aws-mongo-00
ubuntu@ip-172-31-0-117:~/app/backend$ cd /home/ubuntu/app/backend
python3 app.py
* Serving Flask app 'app'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:9000
* Running on http://172.31.0.117:9000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:49.36.25.218 - - [24/Sep/2026 19:26:21] "GET / HTTP/1.1" 200 -
INFO:werkzeug:49.36.25.218 - - [24/Sep/2026 19:26:22] "GET /favicon.ico HTTP/1.1" 404 -
INFO:werkzeug:49.36.25.218 - - [24/Sep/2026 19:26:24] "GET / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [24/Sep/2026 19:26:38] "POST /process HTTP/1.1" 400 -
INFO:werkzeug:127.0.0.1 - - [24/Sep/2026 19:27:04] "POST /process HTTP/1.1" 400 -
INFO:__main__:Data saved for: terra
INFO:werkzeug:127.0.0.1 - - [24/Sep/2026 19:27:14] "POST /process HTTP/1.1" 200 -
```

Step 7: Run Backend as a Background Service

Stop the interactive foreground process and start the Flask application as a persistent background task using `nohup`.

Command:

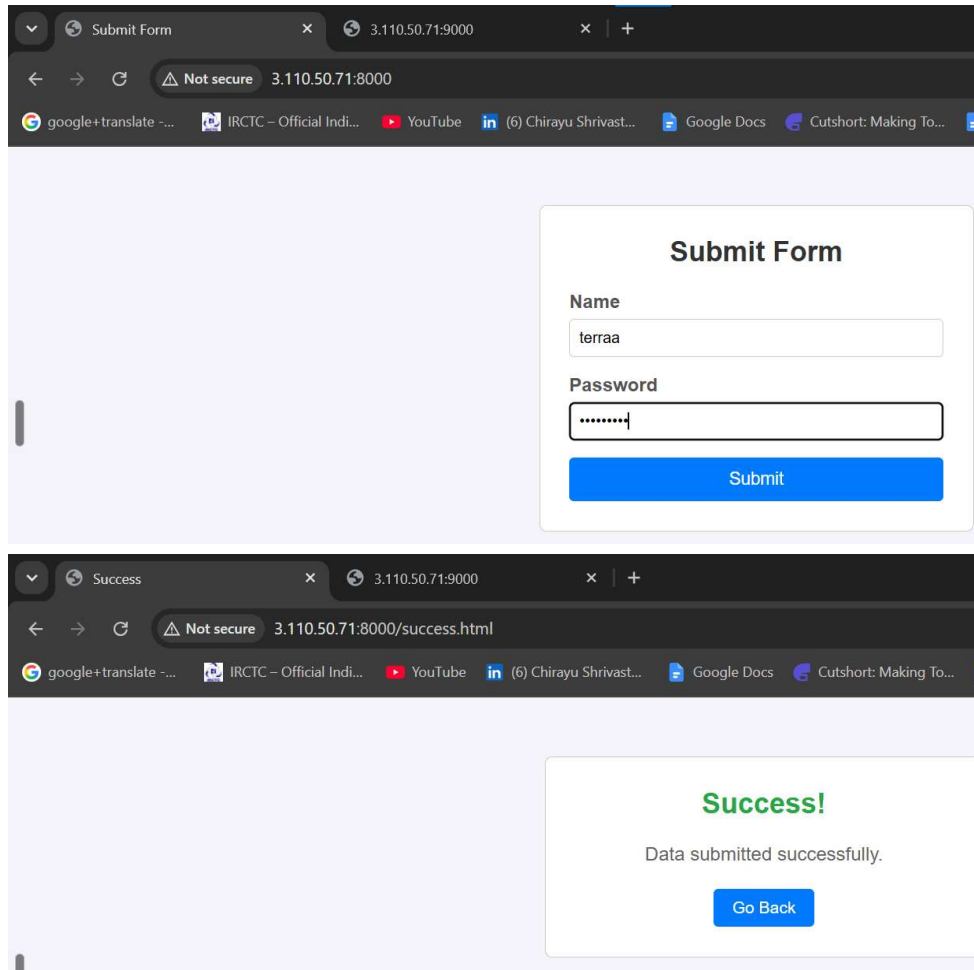
→ `nohup python3 app.py > backend.log 2>&1 &`

Verify both Node (8000) and Python (9000) are actively listening

```
ubuntu@ip-172-31-0-117: ~/app/backend$ sudo ss -tulpn | grep -E '8000|9000'
tcp    LISTEN 0      128          0.0.0.0:9000      0.0.0.0:*        users:((("python3",pid=6077,fd=3))
tcp    LISTEN 0      511          *:8000           *:.*             users:((("node",pid=5858,fd=18))
ubuntu@ip-172-31-0-117:~/app/backend$
```

Step 8: Browser Verification

Validate the entire application flow by submitting a test record from the frontend interface and checking connectivity to MongoDB.



Active Listening Ports:

Command:

→ **sudo ss -tulpn**

```
ubuntu@ip-172-31-0-117:~/app/backend$ sudo ss -tulpn
Netid      State      Recv-Q     Send-Q          Local Address:Port        Peer Address:Port        Process
udp        UNCONN    0           0          127.0.0.53%lo:53          0.0.0.0:*                users:({"systemd-resolve",pid=346,fd=13})
udp        UNCONN    0           0      172.31.0.117%ens5:68      0.0.0.0:*                users:({"systemd-network",pid=344,fd=15})
udp        UNCONN    0           0          127.0.0.1:323            0.0.0.0:*                users:({"chronyd",pid=428,fd=5})
udp        UNCONN    0           0           [::]:323                  [::]:*                   users:({"chronyd",pid=428,fd=6})
tcp        LISTEN    0          4096          127.0.0.53%lo:53          0.0.0.0:*                users:({"systemd-resolve",pid=346,fd=14})
tcp        LISTEN    0          128           0.0.0.0:22                0.0.0.0:*                users:({"ssh",pid=653,fd=3})
tcp        LISTEN    0          128           0.0.0.0:9000              0.0.0.0:*                users:({"python3",pid=6077,fd=3})
tcp        LISTEN    0          128           [::]:22                   [::]:*                   users:({"ssh",pid=653,fd=4})
tcp        LISTEN    0          511              *:8000                    *:*
```

- Removed hardcoded secrets from the Terraform files so nothing sensitive was sitting in version control.
- Updated `.gitignore` to stop tracking `terraform.tfvars`, `*.tfstate`, `.terraform/`, and `.pem` key files.
- Kept the real database credentials only in a local `terraform.tfvars` file, not pushed to GitHub.
- Marked `db_password` and `db_user` as `sensitive = true` in `variables.tf` so they don't show up in logs or terminal output.
- Updated the `user_data` script in `main.tf` to pull in environment variables at instance startup, and locked down the `.env` file with `chmod 600` so only the right user can read it.
- Pushed all these changes to GitHub safely.

Successfully verified the deployment state with zero drift, securely provisioning the EC2 instance with active public IP **13.233.193.252**.

```
Everything up-to-date
→ part1-single-ec2 git:(main → terraform apply
^Ldata.aws_ami.ubuntu: Reading...
aws_security_group.app_sg: Refreshing state... [id=sg-0f51689b6972c32b0]
data.aws_ami.ubuntu: Read complete after 0s [id=ami-07dfe2c11d33a069a]
aws_instance.app_server: Refreshing state... [id=i-0af84b2b9ae01827f]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

ec2_instance_id = "i-0af84b2b9ae01827f"
ec2_public_ip = "13.233.193.252"
→ part1-single-ec2 git:(main →
→ part1-single-ec2 git:(main → |
```

Explanation: Part2-separate-ec2

Step 1: Created a dedicated working directory **part2-separate-ec2** to maintain isolation from the single-instance setup.

Step 2: Created the required Terraform configuration files (**main.tf**, **variables.tf**, **outputs.tf**, **terraform.tfvars**, and **.gitignore**) in the **part2-separate-ec2** directory to manage custom networking, sensitive credentials, and the two-tier infrastructure.

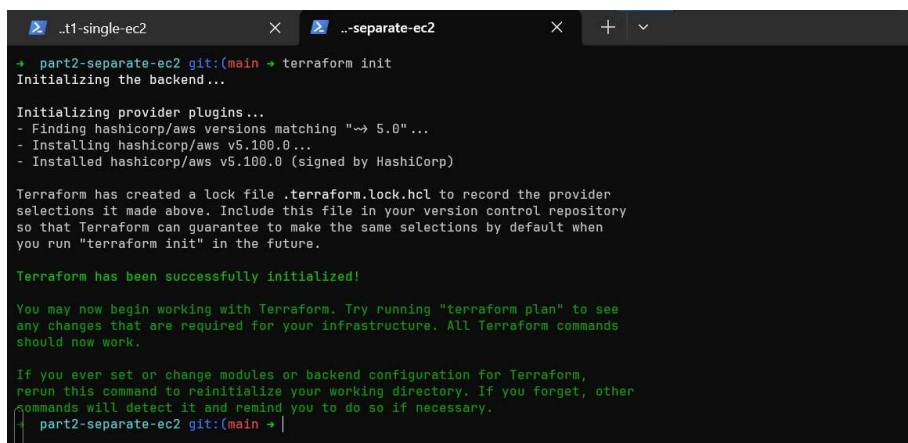
File-by-File Breakdown:

- **main.tf**: Defines the custom VPC, subnets, route tables, security groups, and both backend and frontend EC2 instances.
- **variables.tf**: Declares parameterized input variables for the AWS region, instance type, and masked database secrets.
- **terraform.tfvars**: Stores local, non-version-controlled environment values and sensitive passwords.
- **outputs.tf**: Exports the public IPs of both instances upon successful provisioning.
- **.gitignore**: Prevents sensitive credential files and Terraform state from being pushed to Git.

Step 3: Initialized the AWS Terraform provider and deployed the complete infrastructure, including the custom VPC, subnets, security groups, and both EC2 instances.

Command:

- **terraform init**
- **terraform validate**



```
part2-separate-ec2 git:(main) terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0" ...
- Installing hashicorp/aws v5.100.0 ...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
part2-separate-ec2 git:(main) |
```

Step 4: Verified deployment by accessing the Flask backend on port 9000 and the Express frontend on port 8000 via their respective public IP addresses.

Command:

→ terraform apply

```
Changes to Outputs:
+ backend_public_ip = (known after apply)
+ frontend_public_ip = (known after apply)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.app_vpc: Creating...
aws_vpc.app_vpc: Still creating... [00m10s elapsed]
aws_vpc.app_vpc: Creation complete after 11s [id=vpc-09f7f7bc12fa92565]
aws_internet_gateway.igw: Creating...
aws_subnet.public_subnet: Creating...
aws_security_group.frontend_sg: Creating...
aws_security_group.backend_sg: Creating...
aws_internet_gateway.igw: Creation complete after 0s [id=igw-0329c8507a37a535c]
aws_route_table.public_rt: Creating...
aws_route_table.public_rt: Creation complete after 1s [id=rtb-0b8fbc6c9bb9469bc]
aws_security_group.frontend_sg: Creation complete after 3s [id=sg-07d87f428aff9c4e8]
aws_security_group.backend_sg: Creation complete after 3s [id=sg-0f5860a6079d12598]
aws_subnet.public_subnet: Still creating... [00m10s elapsed]
aws_subnet.public_subnet: Creation complete after 11s [id=subnet-0589f140fd9b321eb]
aws_route_table_association.public_assoc: Creating...
aws_instance.frontend_server: Creating...
aws_instance.backend_server: Creating...
aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-0640f788efbcbef73]
aws_instance.frontend_server: Still creating... [00m10s elapsed]
aws_instance.backend_server: Still creating... [00m10s elapsed]
aws_instance.frontend_server: Creation complete after 13s [id=i-0d64a175797323a4c]
aws_instance.backend_server: Creation complete after 13s [id=i-0198578633d2971e7]

Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

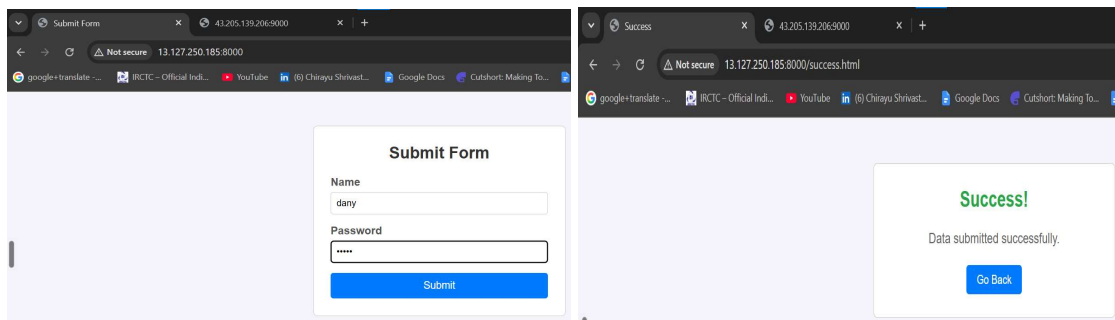
Outputs:

backend_public_ip = "43.205.139.206"
frontend_public_ip = "13.232.204.162"
+ part2-separate-ec2 git:(main) →
```

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 I
single-ec2-fullstack-app	i-0f4e7d8bad29ed02	Stopped	t3.micro	-	ap-south-1b	-	-	-	-
Flask-Express-Server	i-0af84b2b9ae01827f	Stopped	t3.micro	-	ap-south-1b	-	-	-	-
Flask-Backend-Server	i-0198578633d2971e7	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-43-205-139-206.ap...	43.205.139.206	-	-
Express-Frontend-Server	i-0d64a175797323a4c	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-13-232-204-162.ap...	13.232.204.162	-	-

Step 5: Troubleshooting & Inter-Tier Connectivity Configuration

- **Backend Initialization:** Resolved initial startup issues on the Flask server by manually installing Python dependencies, creating the `.env` configuration file with MongoDB credentials, and starting the API service on port 9000 (`43.205.139.206:9000`).
- **Frontend-to-Backend Routing:** Identified that the Express server defaulted to `localhost:9000`[cite: 6]. Configured the `BACKEND_URL` environment variable to point to the backend server endpoint (`[http://43.205.139.206:9000](http://43.205.139.206:9000)`) to enable seamless inter-tier communication.
- **File Permissions & Service Restart:** Corrected ownership permissions (`chown`) on the frontend directory to resolve log write restrictions, followed by restarting the Node.js application with the updated backend environment variable.
- **End-to-End Verification:** Validated complete two-tier connectivity by accessing the Express UI (`13.127.250.185:8000`), submitting form data, and successfully persisting records into the MongoDB database via the Flask backend.
- Added `dnspython` to `requirements.txt` because `PyMongo` requires it to resolve DNS SRV records for MongoDB Atlas `mongodb+srv://` connection strings.



→ Current Active IP Addresses

Backend: <http://13.233.90.150:9000/>

Frontend: <http://15.206.116.104:8000/>

Explanation: Part3-docker-ecs

Step 1: Backend Docker Image Build & Push

- Logged in to Amazon ECR using AWS CLI so Docker could push images.
- Built the Flask backend Docker image as **flask-backend**.
- Tagged the image with the ECR repository URL (**flask-backend:latest**).
- Pushed the image to the **flask-backend** ECR repository successfully.

```

└─ part3-docker-ecs git:(main) # aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 602167898067.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
└─ part3-docker-ecs git:(main) # cd .././backend
└─ backend git:(main) # docker build -t flask-backend .
[+] Building 58.9s (11/11) FINISHED
    => [internal] load build definition from Dockerfile
    => transferring dockerfile: 347B
    => [internal] load metadata for docker.io/library/python:3.10-slim
    => [internal] load .dockerignore
    => transferring context: 2B
    => [1/5] FROM docker.io/library/python:3.10-slim@sha256:08e2c2a09901a007790cd17a0d3313c3f0d0909e19dc70f152c09f811ae9e
    => resolve docker.io/library/python:3.10-slim@sha256:08e2c2a09901a007790cd17a0d3313c3f0d0909e19dc70f152c09f811ae9e
    => sha256:1190f607cc04f2e2a214c2b444026c316a1a62cc5a0b707161a882a7fee 6B / 1.29MB
    => sha256:794247a2db5c2f2e2a214c2b444026c316a1a62cc5a0b707161a882a7fee 13.42MB / 13.42MB
    => sha256:c01be171a50baa48719f0eac842a9f8a420cda18917baa6716a4a035f1 24MB / 24MB
    => sha256:ec53101a559bdc007b39802578007eade876ebccf357c3201a7a299111a08a 29.84MB / 29.84MB
    => extracting sha256:ec53101a559bdc007b39802578007eade876ebccf357c3201a7a299111a08a
    => extracting sha256:1190f607cc04f2e2a214c2b444026c316a1a62cc5a0b707161a882a7fee
    => extracting sha256:794247a2db5c2f2e2a214c2b444026c316a1a62cc5a0b707161a882a7fee
    => extracting sha256:c01be171a50baa48719f0eac842a9f8a420cda18917baa6716a4a035f1
    => [internal] load build context
    => transferring context: 27.84MB
    => [2/5] WORKDIR /app
    => [3/5] COPY requirements.txt .
    => [4/5] Run pip install --no-cache-dir -r requirements.txt
    => [5/5] RUN apt-get update && apt-get install -y curl
    => [6/5] COPY . .
    => exporting to image
    => exporting layers
    => exporting manifest sha256:1222a4ff311859779e0cf0280e50f0f7e9093c9a476a0007e11810a3098
    => exporting config sha256:f31ba02871718a4217e195f3a2275c2b0a40f70a0a5374a308b007e19002094
    => exporting attestation manifest sha256:c1f02a98a005e108a553a4330902e180ff4a0a3f1e9900c0f33d71409d7
    => exporting manifest list sha256:c37b7a7f1d0576a3d174c4be8a2e8f0947702ed37376d302c0289a2bb28e775d
    => naming to docker.io/library/flask-backend:latest
    => unpacking to docker.io/library/flask-backend:latest
└─ backend git:(main) # docker tag flask-backend:latest 602167898067.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest
└─ backend git:(main) # docker push 602167898067.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest
The push refers to repository [602167898067.dkr.ecr.ap-south-1.amazonaws.com/flask-backend]
09fa9ea7ce93: Pushed
64336fa355b3: Already exists
1189ef607cc0: Pushed
ecc510c1e359: Pushed
79f4247a2db5: Pushed
c01be171a5b0: Pushed
b4bde3ba35ba: Pushed
d879a8552db2: Pushed
8b0810f21de5: Pushed
216ae7b3b9a: Pushed
350bf9cf2cd1: Pushed
latest: digest: sha256:037e7ef1d0576a3d174c4be8a2e8f0947702ed37376d302c0289a2bb28e775d size: 856
└─ backend git:(main) #
```

Step 2: Frontend Docker Image Build & Push

- Built the frontend Docker image as **flask-frontend**.
- Tagged the image with the ECR repository URL (**flask-frontend:latest**).
- Pushed the image to the **flask-frontend** ECR repository successfully.

```
.rayu/frontend x ubuntu@ip-10-0-1-138: ~/app x + v
+ frontend git:(main) - docker build -t express-frontend .
[+] Building 15.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [internal] load .dockerignore
=> [internal] load context: 2B
=> [internal] load build context
=> [internal] transferring context: 4.69MB
=> [1/6] FROM docker.io/library/node:18-alpine@sha256:866d21d66304c28f43e60c9f312f249011d1189455880a35c0b0c4b8ca090fe
=> resolve docker.io/library/node:18-alpine@sha256:866d21d66304c28f43e60c9f312f249011d1189455880a35c0b0c4b8ca090fe
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY package*.json ./
=> CACHED [4/6] RUN npm install
=> CACHED [5/6] RUN npm add --no-cache curl
=> [6/6] COPY
=> exporting to image
=> exporting layers
=> exporting manifest sha256:09130bc82127a0d57c9e8f4dc97ad0c137081190b3aeeef4122a7ee00e0a8e
=> exporting config sha256:4e129f8a9494f29419c767d2d0bc071f030843b0d0c7c0b15ee762d0d9f3
=> exporting attestation manifest sha256:b5eb2113ea18863985a251b4c32602af0ae948907cd7e0880bd51f0c35e20fa3
=> exporting manifest list sha256:11d08008a420b23016591cd005f3e9f16327ed6ed14655c709ead0d0c82826
=> sending to docker.io/library/express-frontend:latest
=> unpacking to docker.io/library/express-frontend:latest
+ frontend git:(main) - docker tag express-frontend:latest 602167898067.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest
+ frontend git:(main) - docker push 602167898067.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest
The push refers to repository [602167898067.dkr.ecr.ap-south-1.amazonaws.com/express-frontend]
f170a3a23973: Pushed
64136fa355b3: Layer already exists
6d71d9e834b5: Layer already exists
f13232174bc9: Layer already exists
1e5a4c89cee5: Layer already exists
2ae60b7f50e8: Layer already exists
8f402ac07f33: Layer already exists
25ff2d635441: Layer already exists
774bd125aee2: Layer already exists
f0b3a7674c3b: Layer already exists
1838dcdbd0fe: Pushed
latest: digest: sha256:2140b0d4623bb23b14591cd0a5f3e9f143296d4ed41465547099aa8a5dc82826 size: 850
+ frontend git:(main) -
```

Step 3: Terraform Setup for ECS Deployment (part3-docker-ecs)

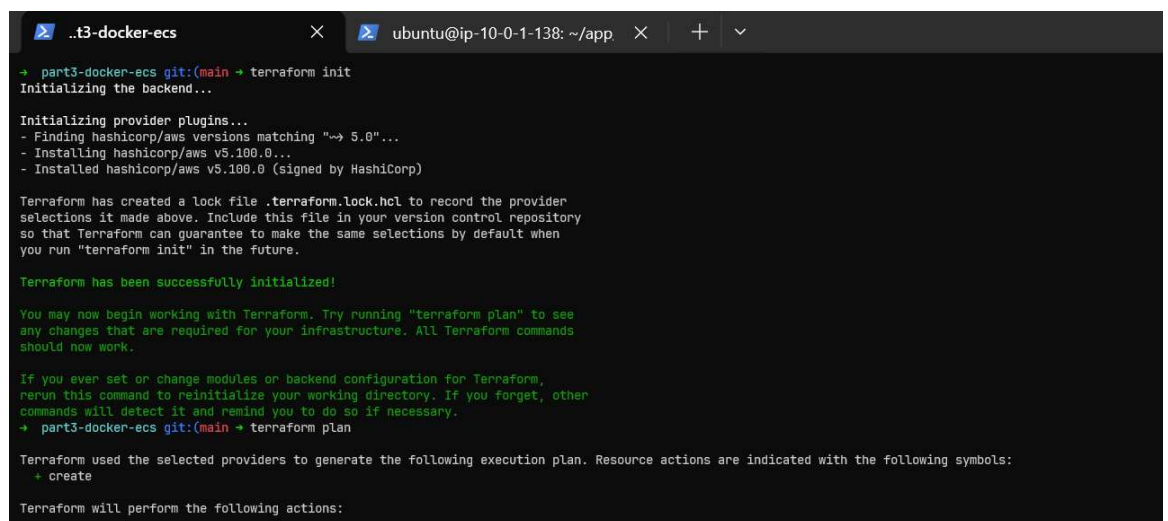
- **main.tf** has the main infrastructure code that creates the AWS resources (ECS cluster, task definitions, services) using the Docker images pushed to ECR.
- **variables.tf** declares the input variables used in the project, like region and image names.
- **terraform.tfvars** provides the actual values for those variables.
- **outputs.tf** shows useful results after deployment, like the load balancer URL.

Step 4: Deploy with Terraform and Verify

- Move into the **terraform/part3-docker-ecs** folder where the Terraform files are.
- Run **terraform init** to download the AWS provider and set up Terraform.
- Run **terraform plan** to preview the resources that will be created, then **terraform apply -auto-approve** to create them without a confirmation prompt.
- After apply finishes, copy the **frontend_url** from the outputs, open it in a browser, and submit the form to verify the app works end to end.

Commands:

- **cd terraform/part3-docker-ecs**
- **terraform init**
- **terraform plan**
- **terraform apply -auto-approve**

A terminal window with a dark background. The title bar shows two tabs: '..t3-docker-ecs' and 'ubuntu@ip-10-0-1-138: ~/app'. The terminal output shows the execution of 'terraform init'. It starts with 'part3-docker-ecs git:(main) terraform init', followed by 'Initializing the backend...' and 'Initializing provider plugins...'. It then lists the steps for finding and installing the AWS provider (v5.100.0). A message states that a lock file '.terraform.lock.hcl' has been created. A green message 'Terraform has been successfully initialized!' is displayed. Further instructions are provided about working with Terraform and running 'terraform plan'. The prompt returns to 'part3-docker-ecs git:(main) terraform plan'. At the bottom, it shows the start of the execution plan generation, listing '+ create' as a resource action.

```
..t3-docker-ecs  X  ubuntu@ip-10-0-1-138: ~/app  X  +  v
→ part3-docker-ecs git:(main) terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
→ part3-docker-ecs git:(main) terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:
```

Terraform Apply and Outputs

terraform apply created 22 AWS resources successfully and returned the ALB DNS name, backend URL (port 9000), and frontend URL (port 8000).

```
Apply complete! Resources: 22 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "part3-alb-252094985.ap-south-1.elb.amazonaws.com"
backend_url = "http://part3-alb-252094985.ap-south-1.elb.amazonaws.com:9000"
frontend_url = "http://part3-alb-252094985.ap-south-1.elb.amazonaws.com:8000"
→ part3-docker-ecs git:(main) → |
```

(ECS + ALB) - Main Proofs:

1. AWS ECS Cluster Console: **part3-ecs-cluster** showing 2 Active Services and 2 Running Tasks.

The screenshot shows the AWS ECS Cluster Console for the cluster named 'part3-ecs-cluster'. The cluster is in an 'Active' state. The 'Services' tab is selected, showing two active services: 'express-frontend-service' and 'flask-backend-service'. Both services are in a 'Running' state with 2 tasks each. The 'Tasks' tab is also visible, showing the tasks for each service.

Service name	ARN	Status	Schedu...	Launch...	Task definition
express-frontend-service	arn:aws:ecs:ap-south-1:602167898067:service/part3-ecs-cluster/express-frontend-service	Active	Replica	Fargate	express-frontend-t...
flask-backend-service	arn:aws:ecs:ap-south-1:602167898067:service/part3-ecs-cluster/flask-backend-service	Active	Replica	Fargate	flask-backend-task...

2. Target Groups Console (Targets tab): View showing the **Healthy** status of **part3-backend-tg** and **part3-frontend-tg**.

The screenshot shows the AWS Target Groups console. The left sidebar contains navigation links for EC2, IAM, VPC, Elastic Container Registry, Elastic Container Service, and CloudFormation. The main content area is titled 'Target groups (1/2)' and shows a table of target groups. The 'part3-backend-tg' target group is selected, and its 'Targets' tab is active. The 'Registered targets' section shows one target with IP address 10.0.1.58, port 9000, and health status 'Healthy'.

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
☒ part3-backend-tg	arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/part3-backend-tg/123456789012	9000	HTTP	IP	part3-alb	vpc-0b92aef4ba30e0e2b
☐ part3-frontend-tg	arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/part3-frontend-tg/123456789012	8000	HTTP	IP	part3-alb	vpc-0b92aef4ba30e0e2b

Target group: part3-backend-tg

Registered targets (1)

IP address	Port	Zone	Health status	Health status details	Administrative override	Override details	Anomaly detection re...
☐ 10.0.1.58	9000	ap-south-1...	Healthy	-	No override	No override is currently active on target	Normal

The screenshot shows the AWS Target Groups console. The left sidebar contains navigation links for EC2, IAM, VPC, Elastic Container Registry, Elastic Container Service, and CloudFormation. The main content area is titled 'Target groups (1/2)' and shows a table of target groups. The 'part3-frontend-tg' target group is selected, and its 'Targets' tab is active. The 'Registered targets' section shows one target with IP address 10.0.2.93, port 8000, and health status 'Healthy'.

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
☐ part3-backend-tg	arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/part3-backend-tg/123456789012	9000	HTTP	IP	part3-alb	vpc-0b92aef4ba30e0e2b
☒ part3-frontend-tg	arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/part3-frontend-tg/123456789012	8000	HTTP	IP	part3-alb	vpc-0b92aef4ba30e0e2b

Target group: part3-frontend-tg

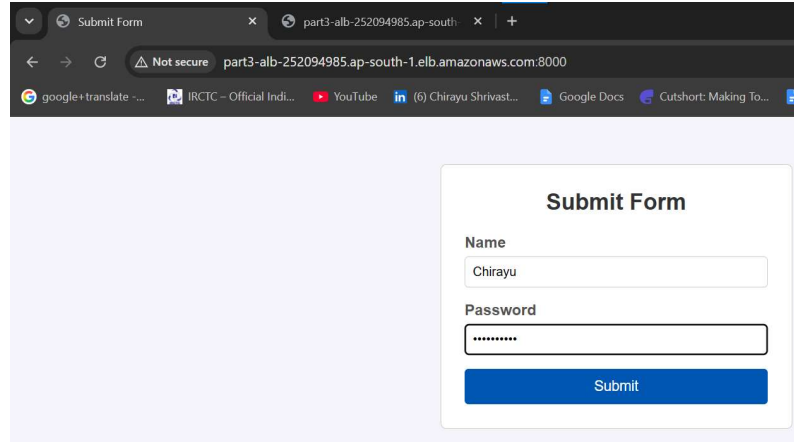
Registered targets (1)

IP address	Port	Zone	Health status	Health status details	Administrative override	Override details	Anomaly detection re...
☐ 10.0.2.93	8000	ap-south-1...	Healthy	-	No override	No override is currently active on target	Normal

3. Browser Output:

- Frontend form on the ALB URL

(<http://part3-alb-252094985.ap-south-1.elb.amazonaws.com:8000/>).



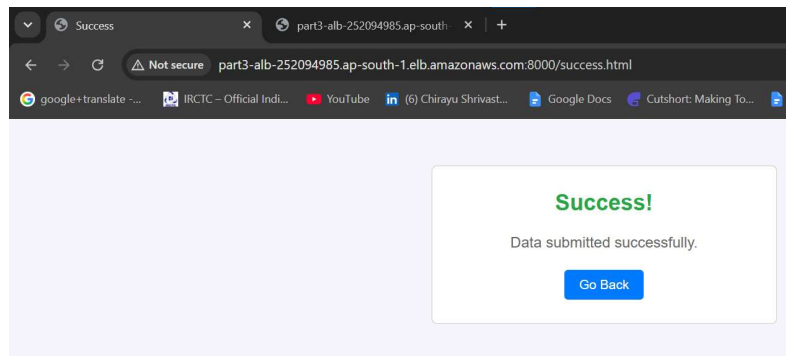
Submit Form

Name
Chirayu

Password

Submit

- Success page (</success.html>) after submitting the form.



4. MongoDB Atlas Database Console: The saved document in the database collection (proof that data storage is working).

